

DEEP LEARNING

Report 3: Logistic Regression

Student: Vu Ha Vy – 2540112

Implementation

Loss function

Unlike linear regression, logistic regression predicts a probability using the sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$, where $z = w_1x_1 + w_2x_2 + w_0$. The model is trained by maximizing the log-likelihood, which is equivalent to minimizing the binary cross-entropy loss:

$$J(w_0, w_1, w_2) = -\frac{1}{N} \sum_{i=1}^N \left[y_i \cdot z_i - \log(1 + e^{z_i}) \right]$$

```
def loss(data, w0, w1, w2):
    N = len(data)
    total = 0.0

    for x1, x2, yi in data:
        y_hat = w1 * x1 + w2 * x2 + w0
        z = yi * y_hat - math.log(1 + math.exp(y_hat))
        total += z

    return -(1 / N) * total
```

Gradient descent

The weights are updated iteratively using the partial derivatives of J . The error at each point is $\hat{y}_i - y_i$, where $\hat{y}_i = \sigma(z_i)$ is the predicted probability:

$$\frac{\partial J}{\partial w_0} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i) \quad \frac{\partial J}{\partial w_1} = \frac{1}{N} \sum_{i=1}^N x_{1i} (\hat{y}_i - y_i) \quad \frac{\partial J}{\partial w_2} = \frac{1}{N} \sum_{i=1}^N x_{2i} (\hat{y}_i - y_i)$$

```
def gradient_descent(data, lr, thres):
    w0 = 0.0
    w1 = 0.0
    w2 = 0.0

    N = len(data)
    i = 0
    J_prev = float('inf')

    while True:
        i += 1

        J = loss(data, w0, w1, w2)

        if abs(J - J_prev) < thres:
            print(f"Converged at iteration {i}, J = {J}")
            break

        J_prev = J

        print(f"{i} | {w0:.4f} | {w1:.4f} | {w2:.4f} | {J:.4f}")

        dw0 = 0.0
        dw1 = 0.0
        dw2 = 0.0
```

```

for x1, x2, yi in data:
    z = w1 * x1 + w2 * x2 + w0
    prediction = sigmoid(z)
    error = prediction - yi

    dw0 += error
    dw1 += x1 * error
    dw2 += x2 * error

dw0 /= N
dw1 /= N
dw2 /= N

w0 = w0 - lr * dw0
w1 = w1 - lr * dw1
w2 = w2 - lr * dw2

return w0, w1, w2

```

Dataset

The dataset `loan2.csv` contains 19 data points with two features, Experience (years) and Salary (million), and a binary label indicating whether the loan was approved (1) or refused (0).

Experience (year)	Salary (million)	Loan
3.00	4	1
2.50	4	1
1.00	4	0
2.50	5	1
2.00	5	1
1.50	5	0
0.50	5	0
1.75	6	1
0.25	6	0
1.00	7	1
0.25	7	0
0.20	7	0
0.15	7	0
2.00	8	1
1.00	8	0
0.15	8	0
0.10	8	0
0.50	9	1
1.00	10	1

Table 1: Contents of `loan2.csv`

Results

With learning rate $r = 0.1$ and threshold $t = 10^{-6}$, the algorithm converged after 13277 iterations with final loss $J = 0.2058$. The learned model is:

$$\hat{y} = \sigma(-13.5672 + 4.5400 \cdot x_1 + 0.2058 \cdot x_2)$$

where x_1 is Experience, x_2 is Salary, and $\sigma(z) = \frac{1}{1+e^{-z}}$.

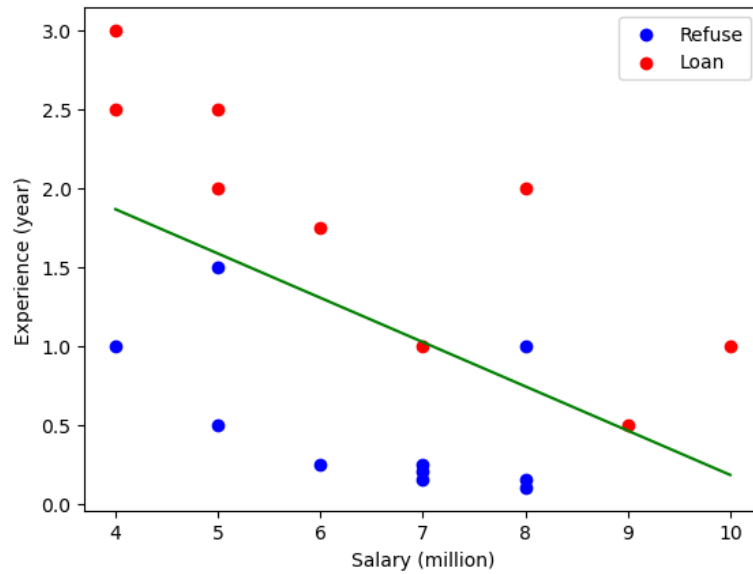


Figure 1: Decision boundary (green) separating approved loans (red) from refused loans (blue), plotted over Salary vs. Experience.

Effects of different learning rates

Table below compares three learning rates with the same threshold $t = 10^{-6}$. A larger r converges faster and also reaches a lower loss, because bigger steps escape flat regions of the loss surface more quickly. However, setting r too large risks overshooting and diverging. Here $r = 0.1$ gives the best result, converging in 13277 iterations with the lowest loss of 0.2058.

Learning rate r	Iterations	Final loss J	w_0	w_1	w_2
0.001	76,147	0.3460	-3.1537	2.1585	0.1413
0.01	41,356	0.2359	-8.7517	3.3970	0.7597
0.1	13,277	0.2058	-13.5672	4.5400	0.2058

Table 2: Comparison of learning rates (threshold $t = 10^{-6}$)